Token-World: Direct World Model Simulator for Vision-Language-Action Models

Anonymous Author(s)
Affiliation
Address
email

Abstract

World-model simulators for vision-language-action (VLA) systems often predict future RGB frames and then re-encode them into policy inputs. This indirect decode–encode interface creates a representation mismatch between the simulator and the downstream policy, adding computational overhead and potential token-level information loss to every imagined interaction.To address this bottleneck, we introduce **Token-World**, an action-conditioned world model that operates directly in VLA visual token space. Token-World uses a spatio-temporal multimodal Transformer with an x_0-parameterized flow-matching objective to model high-dimensional semantic token dynamics, together with a GRU-based recurrent state that preserves past interaction context during long-horizon rollout. Experiments evaluate Token-World along two complementary axes: for token-dynamics modeling, the 2D flow-matching Transformer improves token prediction accuracy over deterministic Transformer-based baselines; for the simulation interface, direct token-space rollout outperforms RGB-based world models in long-horizon prediction accuracy, inference efficiency, and rollout-based success classification.

1 Introduction

World models [18] offer a promising route toward scalable embodied learning by serving as learned simulators of action-conditioned environment dynamics [14, 19, 20, 51, 53]. Instead of executing every behavior in the physical world, an agent can roll out future observations inside the model, which is especially valuable in robotics where real-world interaction is costly and slow.[13, 51]. Recent work has therefore explored world models as simulators from several perspectives: using predicted futures to evaluate policies[16, 33, 34, 38, 47], synthesizing additional interaction data for policy learning[17, 27, 45, 49], and performing reinforcement learning through imagined rollouts[8, 28, 29, 39, 52, 57].

As world models are increasingly used as learned simulators, they are often paired with VLA policies that map visual observations and language instructions to actions [7, 26, 31, 46]. However, existing pipelines often use an indirect simulation interface, where future RGB observations are first predicted by the world model, then re-encoded into VLA visual tokens before being consumed by the VLA policy [8, 16, 17, 27–29, 33, 34, 38, 39, 45, 47, 49, 52, 57]. As world models move toward scalable simulators for VLA agents, this indirect interface becomes a critical bottleneck: the simulator predicts human-viewable pixels, whereas the downstream policy ultimately consumes policy-facing visual tokens.This mismatch is not merely computational: RGB reconstruction encourages the simulator to allocate capacity to visual details that may be irrelevant to the policy, while token-space rollout directly targets the representation used for downstream action generation.

A representation-aligned alternative is to simulate the future directly in the VLA visual-token space consumed by the policy. This direct interface bypasses the decode–encode pipeline, avoiding repeated

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

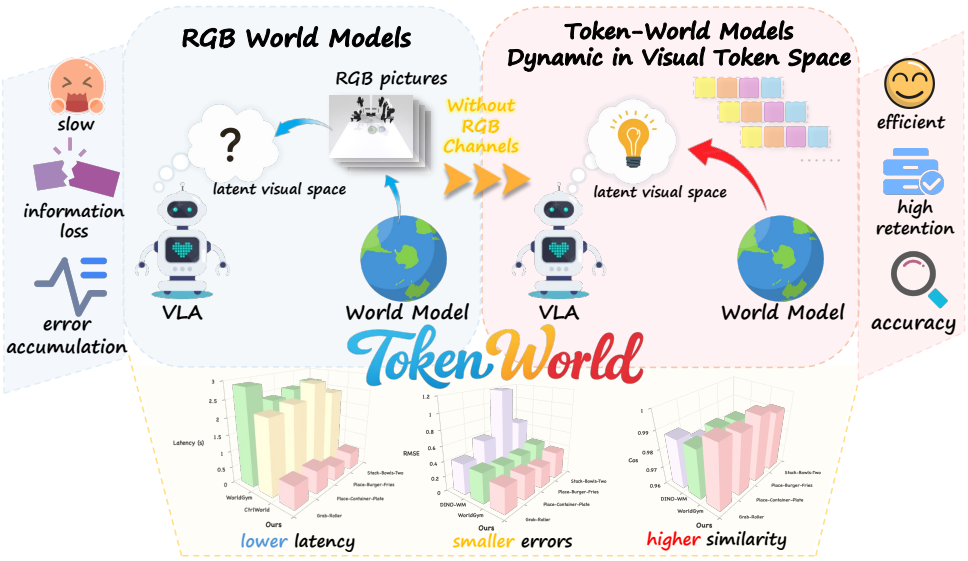


Figure 1: **Token-World simulates directly in VLA visual-token space.** RGB-based simulators predict future frames and re-encode them into VLA visual tokens, introducing a decode–encode interface. Token-World rolls out directly in the policy-facing token space, yielding more faithful token predictions with about $4\times$ lower simulator step time.

reconstruction and re-tokenization during imagined rollouts. However, direct VLA visual-token simulation is non-trivial: these tokens are compact, policy-facing representations [7, 26, 31], and predicting their future evolution requires modeling high-dimensional dynamics while retaining past interaction context [50]. Therefore, both the dynamics architecture and the learning objective must be designed around the specific challenges of token-space rollout.

In this work, we introduce Token-World, a world model simulator that directly operates in VLA visual token space. To model high-dimensional VLA visual token dynamics, we use a spatio-temporal multimodal Transformer [21, 23, 48] that efficiently captures within-frame token interactions and cross-frame temporal evolution. Following [21, 32], Token-World uses x_0-parameterized flow-matching objective to better model the high-dimensional token dynamics. Moreover, we introduce a GRU-based recurrent state [11] to preserve the history-dependent context of VLA visual tokens[19, 20].

Our experiments disentangle two key design choices. First, for token-dynamics modeling, the 2D flow-matching Transformer improves token prediction accuracy over deterministic Transformer-based baselines. Second, for the simulation interface, direct token-space rollout achieves better long-horizon prediction accuracy, higher inference efficiency, and more reliable rollout-based success classification than RGB-based world models. In summary, our contributions are threefold:

- We identify the decode–encode interface as a representation mismatch in world-model simulators for VLA systems, formalize this interface problem in Sec. 3, and propose direct VLA visual-token simulation as a policy-aligned world-model interface.
- We present Token-World, a 2D flow-matching Transformer world model simulator with x_0-parameterized flow-matching objective to model high-dimensional semantic token dynamics, together with a GRU-based recurrent state to retain past interaction context during long-horizon rollout.
- We provide a two-axis empirical validation of Token-World. On the modeling axis, our 2D flow-matching Transformer formulation achieves more accurate VLA visual-token prediction than deterministic Transformer baselines. On the interface axis, direct token-space rollout improves long-horizon prediction accuracy, inference efficiency, and rollout-based success classification over RGB-based world-model simulators.

2 Related Works

2.1 World Models as Learned Simulators

World models learn action-conditioned dynamics and support planning or policy learning through imagined rollouts [14, 18–20, 51, 53]. In robotics, recent work has developed learned simulators for several purposes: evaluating candidate actions or policies with predicted futures [16, 33, 34, 38, 47], generating synthetic interaction data for policy learning [17, 27, 45, 49], and performing reinforcement learning inside learned dynamics models [8, 28, 29, 39, 52, 57]. These methods reduce the need for costly physical interaction and point toward learned world models as a promising path to scalable simulation for embodied agents.

Most of these simulators, however, represent imagined futures as RGB frames. Token-World is complementary to this line of work: instead of proposing a new downstream use of imagined rollouts, it studies the simulation interface itself, directly rolling out future states in the VLA visual-token space consumed by downstream policies.

2.2 Visual Representations for World Models

A key design choice in visual world models is the representation space used for future prediction. A prominent class of recent embodied world models adopts a latent video-generation-style formulation, either by adapting pretrained video generation models or by training similar architectures from scratch. These methods model future dynamics in compressed VAE or video-token latent spaces and decode them back into RGB/RGB-D frames for visual rollout [8, 16, 17, 22, 24, 27–29, 33–35, 38, 45, 47, 49, 52, 55, 57]. Another line of work avoids reconstructing redundant RGB observations and instead models dynamics in representation space. DINO-based methods predict the evolution of pretrained visual features for future scene understanding, planning, or robotic manipulation [3, 30, 56]. In parallel, JEPA-style world models formulate future prediction as latent embedding prediction, enabling efficient planning in learned representation spaces [2, 4, 36, 37, 41, 43]. While both directions move beyond RGB reconstruction, their prediction spaces are usually generic visual representations rather than the policy-facing visual-token space consumed by VLA systems.

Token-World differs from previous work in both representation and dynamics modeling. In representation, we target VLA visual tokens, a policy-facing space directly consumed by downstream VLA agents. In dynamics modeling, we use a 2D flow-matching Transformer architecture with a recurrent state, instead of the deterministic Transformer predictors commonly used in DINO-WM [56], V-JEPA2 [2], and related latent world models.

3 Problem Formulation

Information loss induced by decode–encode interfaces. Many world-model simulators operate in a reconstruction-oriented latent space and interface with a VLA policy through a decode–encode pipeline. Let o_t denote the original observation and let $z_t = E(o_t)$ be the policy-facing VLA visual tokens produced by a frozen VLA visual encoder E. An RGB/VAE-based world model first encodes the observation into a latent variable

$$u_t \sim q_\psi(u_t \mid o_t), \tag{1}$$

and reconstructs an image through a decoder

$$\hat{o}_t = D_\theta(u_t). \tag{2}$$

The VLA policy cannot directly consume u_t or $\hat{o}_t$; instead, the reconstructed image must be re-encoded into visual tokens:

$$\hat{z}_t = E(\hat{o}_t) = E(D_\theta(u_t)). \tag{3}$$

This induces the following data-processing chain:

$$o_t \to u_t \to \hat{o}_t \to \hat{z}_t, \tag{4}$$

while the desired policy-facing token is $z_t = E(o_t)$.

The VAE latent u_t is explicitly rate-limited by its variational objective. For a β-VAE-style formulation,

$$\mathcal{L}_{\mathrm{VAE}} = \mathbb{E}_{q_\psi(u_t|o_t)}\big[-\log p_\theta(o_t \mid u_t)\big] + \beta\,\mathrm{KL}\big(q_\psi(u_t \mid o_t)\,\|\,p(u_t)\big), \tag{5}$$

the KL term upper-bounds the mutual information between the observation and the latent variable:

$$I(o_t; u_t) \le \mathbb{E}_{p(o_t)} \mathrm{KL}\big(q_\psi(u_t \mid o_t) \,\|\, p(u_t)\big). \tag{6}$$

Thus, under a finite-rate latent bottleneck, u_t cannot preserve all information in o_t. Since $\hat{o}_t$ is generated only from u_t, the data processing inequality gives

$$I(o_t; \hat{o}_t) \le I(o_t; u_t), \tag{7}$$

which formalizes the information loss introduced by reconstructing the image from a compressed latent representation.

More importantly for VLA policies, the decode–encode interface can only preserve token information that survives this reconstruction process. Since $z_t = E(o_t)$ and $\hat{z}_t = E(\hat{o}_t)$, we have

$$I(z_t; \hat{z}_t) \le I(z_t; \hat{o}_t) \le I(o_t; \hat{o}_t) \le I(o_t; u_t). \tag{8}$$

Combining the above inequalities, we obtain a direct upper bound on the token information preserved by the decode–encode interface:

$$I(z_t; \hat{z}_t) \le \mathbb{E}_{p(o_t)} \mathrm{KL}\big(q_\psi(u_t \mid o_t) \,\|\, p(u_t)\big). \tag{9}$$

This bound makes the interface bottleneck explicit: the mutual information between the true policy-facing token z_t and the re-encoded token $\hat{z}_t$ cannot exceed the information rate allowed by the VAE latent.

Thus, unless the VAE latent has sufficient capacity to preserve all information needed to recover the VLA token, the decode–encode interface inevitably leaves residual uncertainty about the policy-facing representation. During multi-step imagined rollout, the same bottleneck is repeatedly applied, allowing token-level errors to accumulate over time. This motivates bypassing the reconstruction interface altogether and directly modeling future states in the VLA visual token space.

Direct VLA visual-token simulation. To avoid the intermediate reconstruction bottleneck, Token-World directly models future dynamics in the VLA visual-token space. At each timestep, we treat the VLA visual tokens and proprioceptive state as the world-model state,

$$s_t = (z_t, p_t), \tag{10}$$

where z_t denotes the policy-facing visual tokens and p_t denotes the proprioceptive state. Given the past state history $s_{\le t}$ and the current action a_t, Token-World parameterizes a one-step state dynamics model:

$$p_\theta\big(z_{t+1}, p_{t+1} \mid z_{\le t},\, p_{\le t},\, a_t\big)\,. \tag{11}$$

Long-horizon rollout is then obtained autoregressively by repeatedly applying the one-step model under a future action sequence $a_{t:t+H-1}$:

$$p_\theta(z_{t+1:t+H}, p_{t+1:t+H} \mid z_{\le t},\, p_{\le t},\, a_{t:t+H-1}) = \prod_{k=0}^{H-1} p_\theta(z_{t+k+1}, p_{t+k+1} \mid \hat{z}_{\le t+k},\, \hat{p}_{\le t+k},\, a_{t+k})\,, \tag{12}$$

where $\hat{z}_{\le t+k}$ and $\hat{p}_{\le t+k}$ denote the rollout history formed by appending previously predicted visual tokens and proprioceptive states. The predicted visual tokens remain in the same visual-token space as the downstream VLA policy. Unlike RGB-based simulators, this direct interface avoids decoding latent states into RGB observations and re-encoding them with the VLA visual tokenizer.

4 Token-World

4.1 Overview

Token-World is an action-conditioned world model that directly operates in the VLA visual-token space. Following [21], we implement Token-World as a spatio-temporal multimodal Transformer [48], where each block factorizes modeling into within-frame spatial attention and causal temporal attention [23].To retain interaction context beyond the finite attention window, we further insert a GRU-based temporal state [11] along the temporal axis.

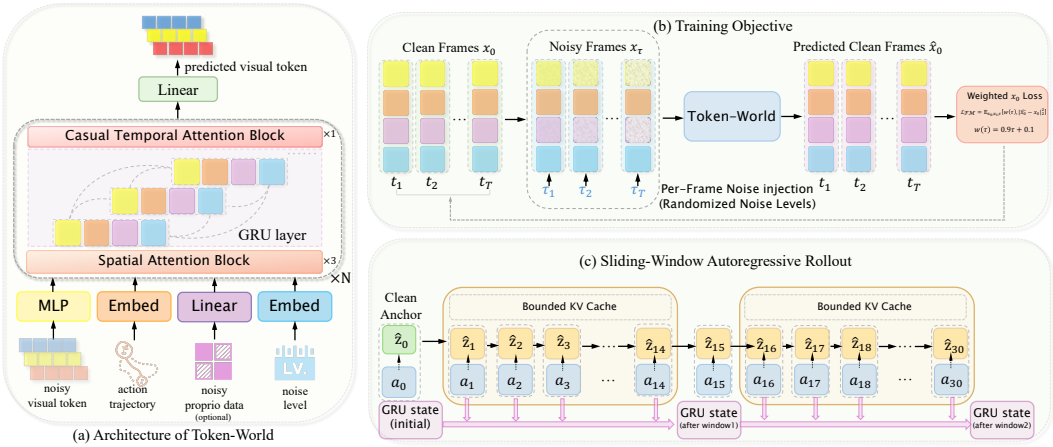


Figure 2: **Token-World overview. (a)** Architecture: a spatio-temporal multimodal Transformer models VLA visual-token dynamics, with a GRU state carrying context beyond the finite attention window. **(b)** Training: x_0 flow matching predicts clean future tokens from frame-level noisy inputs. **(c)** Inference: sliding-window autoregressive rollout generates future tokens directly in the VLA visual-token space.

We adopt an x_0-parameterized flow-matching objective to better model high-dimensional VLA visual token dynamics[32]. In implementation, we train the model through a shortcut forcing objective [9, 15, 21] to improve robustness under autoregressive generation and decrease inference time.

At inference time, Token-World performs windowed autoregressive rollout [9, 24, 25, 49]. Starting from a clean anchor token, the model predicts future VLA visual tokens step by step within a finite context window; when the window reaches its maximum length, the last denoised prediction is used as the anchor for the next window. This allows Token-World to produce long-horizon rollouts entirely in the VLA visual-token space while keeping the temporal context and memory cost bounded.

4.2 Architecture of Token-World

Spatio-Temporal Multimodal Transformer. Following [21], we organize the input as a tensor of shape (B, T, S, D), where T is the rollout-window length, S is the number of multimodal tokens per timestep, and D is the hidden dimension. At each timestep, VLA visual tokens are first projected into the model dimension, while proprioceptive states, actions, and noise level embeddings are mapped into the same token space. These tokens are concatenated to form a multimodal token set for each frame, as shown in Figure 2 (a).

Each Transformer block follows a causal spatio-temporal design over space and time. Tokens within the same frame can attend to each other, while temporal attention is causally masked so that each frame only attends to past context. To reduce the cost of dense attention over all space-time tokens, we decompose attention into space-only and time-only layers [23], and apply temporal attention sparsely across the network [21]. We use a standard pre-norm Transformer implementation with RMSNorm [54], RoPE [42], SwiGLU [40], QK normalization [12], attention logit soft-capping [5, 44], and grouped-query attention (GQA) [1] for stable and efficient training with KV caching.

GRU-based Recurrent State. The spatio-temporal Transformer captures dependencies within a finite attention window. During long-horizon rollout, this window is repeatedly shifted forward, causing earlier interaction context to fall outside the temporal attention range. We therefore introduce a GRU-based [11] recurrent state along the temporal axis. Unlike KV cache, which grows with the number of cached timesteps, the GRU state provides a fixed-size memory that summarizes past token-action context across windows.

Concretely, the hidden representation produced during temporal modeling is used to update a recurrent state that summarizes past visual-token, proprioceptive, and action context. This state is then fed back into the transition model as compact temporal memory. Unlike the temporal KV cache, which

stores token-level keys and values within the current attention window, the GRU state has fixed size and can carry interaction context across windows without growing with rollout length.

4.3 Training Objective and Autoregressive Rollout

Training Objective. Following [21, 32], we train the transition model with an x_0-parameterized flow-matching objective in the VLA visual token space, as shown in Figure 2 (b). Let $\mathbf{x}_0$ denote the clean target future world-model state, as defined in Sec. 3, and let $\mathbf{x}_1 \sim \mathcal{N}(0, \mathbf{I})$ denote Gaussian noise. For a flow time $\tau \in (0, 1]$, we construct

$$\mathbf{x}_\tau = (1 - \tau)\mathbf{x}_0 + \tau\mathbf{x}_1. \tag{13}$$

The transition model predicts the clean endpoint

$$\hat{\mathbf{x}}_0 = f_\theta(\mathbf{x}_\tau, \tau, \mathcal{C}), \tag{14}$$

where $\mathcal{C}$ denotes the context tokens, including visual-token history, proprioceptive states, actions, and the recurrent state. Following [21], we use a ramp weighting term to emphasize harder denoising steps with larger noise levels:

$$w(\tau) = 0.9\tau + 0.1. \tag{15}$$

The final training objective is

$$\mathcal{L}_{\mathrm{FM}} = \mathbb{E}_{\mathbf{x}_0, \mathbf{x}_1, \tau}\left[w(\tau)\,\|\hat{\mathbf{x}}_0 - \mathbf{x}_0\|_2^2\right]. \tag{16}$$

In our windowed training setup, we adopt the shortcut forcing objective over rollout windows [9, 15, 21], assigning different noise levels to different rollout steps. This exposes the model to imperfect autoregressive histories during training and enables efficient inference with a reduced number of denoising steps.

Autoregressive Rollout. At inference time, Token-World performs windowed autoregressive rollout rather than denoising an entire future sequence in one pass [9, 24, 25, 49], as shown in Figure 2 (c). Each rollout window starts from a clean anchor token. For the first window, the anchor is the observed token z_t; for later windows, the anchor is the last denoised prediction from the previous window.

Within a window, future tokens are generated step by step. Let s_m be the starting timestep of the m-th window and L be the maximum window length. For the r-th prediction inside this window, Token-World samples

$$\hat{z}_{s_m+r+1} \sim p_\theta(\cdot \mid \tilde{z}_{s_m:s_m+r},\, p_{s_m:s_m+r},\, a_{s_m:s_m+r}), \quad r = 0, \ldots, L-2, \tag{17}$$

where $\tilde{z}_{s_m}$ is the clean anchor token and $\tilde{z}_{s_m+i} = \hat{z}_{s_m+i}$ for $i > 0$ are tokens generated earlier in the same window. After the window reaches its maximum length, the final predicted token is used as the clean anchor for the next window, and rollout continues with a shifted temporal context.

This procedure keeps the temporal attention window bounded while allowing long-horizon simulation. The GRU recurrent state is carried across windows, providing compact memory of earlier interaction context beyond the current attention range.

5 Experiments

5.1 Overview

Our experiments evaluate Token-World along two complementary axes. First, we study the dynamics modeling axis: given the VLA visual-token prediction target, what modeling choices are effective for VLA visual-token dynamics? Second, we study the simulation-interface axis: when the downstream policy consumes VLA visual tokens, does direct token-space simulation provide a better interface than RGB-based decode–encode simulation? We further include qualitative visualizations to analyze how predicted token changes align with ground-truth semantic dynamics over long-horizon rollouts.

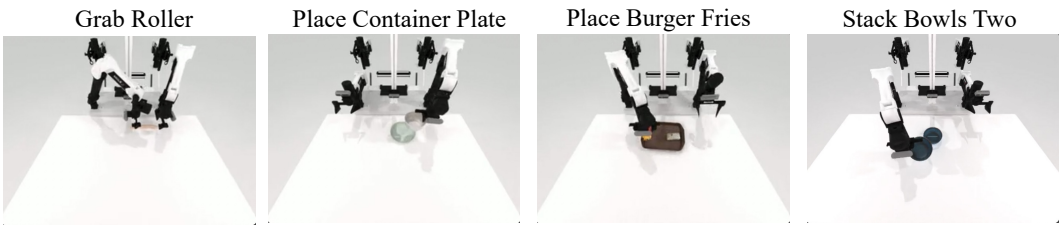


Figure 3: **The four manipulation tasks on RoboTwin benchmark.**

5.2 Experimental Setup

Tasks and data. We evaluate Token-World on four simulated manipulation tasks from RoboTwin [10]: *Grab-Roller*, *Place-Container-Plate*, *Place-Burger-Fries*, and *Stack-Bowls-Two*. For each task, we collect 100 trajectories with a balanced split of 50 successful and 50 failed episodes, except for *Grab-Roller*; details are provided in Appendix A. We split them into 80 training and 20 validation trajectories while preserving the success/failure balance. All world models are trained on the same task data under matched training budgets. We use the Paligemma [6] visual tokenizer from a task-specific π_0 [7] VLA policy as the frozen visual encoder.

Baselines. We group baselines by the two evaluation axes. For *dynamics modeling*, all methods predict VLA visual tokens directly; we compare Token-World with a deterministic Transformer token predictor adapted from DINO-WM [56]. For *simulation interface*, we compare with RGB-based simulators, WorldGym [38] and Ctrl-World [17]. WorldGym predicts VAE latents with a DiT dynamics model and decodes them to RGB, while Ctrl-World adapts a pretrained video diffusion backbone for controllable robot world modeling.

Metrics. We evaluate all models with open-loop action replay. Token fidelity is measured by RMSE and cosine similarity between predicted and ground-truth VLA visual tokens; efficiency is measured by simulator step time; rollout utility is measured by success prediction using a shared latent success classifier, detailed in Appendix D. For simulation-interface evaluation, RGB-based baselines follow their natural decode–encode pipeline: generated RGB frames are re-encoded by the frozen VLA tokenizer before token-space evaluation.

5.3 Evaluating Dynamics Modeling

We first evaluate the *dynamics-modeling axis*: given the same VLA visual-token prediction target, how should the transition dynamics be modeled? We compare Token-World with a deterministic Transformer-based token predictor adapted from DINO-WM [56]. Both methods predict future VLA visual tokens directly; the difference lies in the dynamics modeling choice.

Results. Table 1 and Fig. 4b show that Token-World improves VLA visual-token prediction across all tasks. The gain is most pronounced on long-horizon tasks: on *Stack-Bowls-Two*, RMSE drops from 1.8556 to 0.3712 and cosine similarity rises from 0.4531 to 0.9911 over the deterministic Transformer baseline. Ablations further validate the design choices: removing the GRU degrades rollout fidelity, while replacing x_0-prediction with v-prediction causes a larger drop, with RMSE increasing to 0.5818–0.6507 and cosine similarity falling to around 0.97. These results indicate that Token-World benefits not only from predicting VLA tokens directly, but also from more suitable dynamics modeling in this high-dimensional token space.

5.4 Evaluating the Simulation Interface

We next evaluate the *simulation-interface axis* by comparing Token-World with recent RGB-based world-model simulators [17, 38]. These baselines expose imagined futures as RGB observations and require VLA re-encoding for token-space evaluation, whereas Token-World directly rolls out future states in the VLA visual-token space. Ctrl-World [17] is included only for efficiency because it uses a large pretrained video diffusion backbone, while RMSE/cosine comparisons focus on from-scratch baselines.

Table 1: **Token fidelity under action-replay rollout.** We evaluate two complementary axes: dynamics modeling and simulation interface. For all methods, results are reported in the VLA visual-token space using trajectory-averaged RMSE (lower is better) and cosine similarity (higher is better).

Method	Grab-Roller		Place-Container-Plate		Place-Burger-Fries		Stack-Bowls-Two	
	RMSE↓	Cos↑	RMSE↓	Cos↑	RMSE↓	Cos↑	RMSE↓	Cos↑
Dynamics modeling								
DINO-WM [56]	0.4259	0.9874	0.6670	0.9807	1.7860	0.9741	1.8556	0.4531
Token-World w/o x_0-prediction	0.5818	0.9728	0.5801	0.9755	0.6295	0.9707	0.6507	0.9681
Token-World w/o GRU	0.2676	0.9957	0.3678	0.9910	0.3590	0.9919	0.4069	0.9887
Simulation interface								
WorldGym [38]	0.3947	0.9877	0.3485	0.9910	0.3606	0.9896	0.4102	0.9803
Token-World	**0.2620**	**0.9958**	**0.3236**	**0.9936**	**0.2790**	**0.9960**	**0.3712**	**0.9911**

Table 2: **Rollout-based success prediction under action-replay rollout.** We evaluate two complementary axes: dynamics modeling and simulation interface. We report Acc. (A), Prec. (P), Recall (R), and F1 for success/failure prediction on validation rollouts. Higher is better.

Method	**Grab-Roller**				**Place-Container-Plate**				**Place-Burger-Fries**				**Stack-Bowls-Two**			
	A	P	R	F1	A	P	R	F1	A	P	R	F1	A	P	R	F1
Dynamics modeling																
w/o x_0-prediction	0.67	**1.00**	0.25	0.40	0.50	0	0	0	0.45	0	0	0	0.50	0	0	0
w/o GRU	0.61	0.57	0.50	0.53	0.50	0	0	0	0.50	0	0	0	0.50	0	0	0
Simulation interface																
WorldGym [38]	0.33	0.17	0.13	0.14	0.50	0	0	0	0.50	0	0	0	0.50	0	0	0
Token-World	**0.94**	0.89	**1.00**	**0.94**	**0.60**	**1.00**	**0.20**	**0.33**	**0.70**	**1.00**	**0.40**	**0.57**	**0.65**	**0.71**	**0.50**	**0.59**

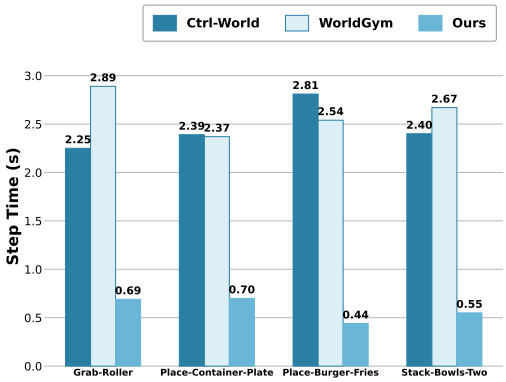
Results. Table 1 and Figure 4 shows that direct token-space simulation improves both rollout fidelity and efficiency. In token-space evaluation, Token-World achieves lower long-horizon RMSE than WorldGym [38] across all four tasks, e.g., reducing RMSE from 0.3947 to 0.2620 on *Grab-Roller*, from 0.3606 to 0.2790 on *Place-Burger-Fries*, and from 0.4102 to 0.3712 on *Stack-Bowls-Two*, while also improving cosine similarity. The efficiency gain is more pronounced: Token-World requires only 0.44–0.70s per simulator step, compared with 2.25–2.81s for Ctrl-World and 2.37–2.89s for WorldGym. This shows that avoiding RGB generation and VLA re-encoding substantially reduces the cost of every imagined interaction.

Table 2 further shows that Token-World improves overall success-prediction accuracy from 0.46 to 0.72 and F1 from 0.05 to 0.63 compared with WorldGym. On *Grab-Roller*, accuracy increases from 0.33 to 0.94 and F1 from 0.14 to 0.94. These results indicate that direct token-space rollouts preserve more task-relevant information than RGB-based decode–encode simulation.

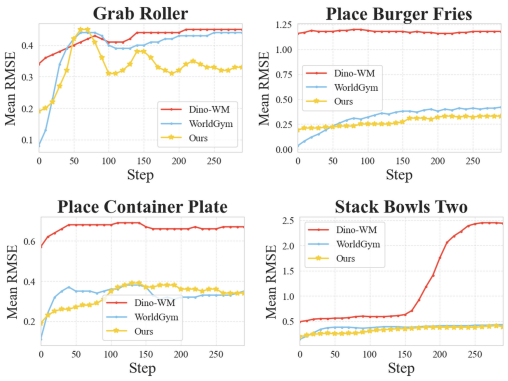
5.5 Qualitative Analysis

Fig. 5 visualizes patch-wise semantic changes in the Paligemma visual-token space. We compute the L2 distance from each timestep to the initial frame on a 16×16 token grid and upsample the map for visualization. The top row shows Token-World predictions, the middle row shows ground-truth token changes, and the bottom row shows RGB frames.

Token-World produces change maps that broadly match the ground-truth spatial patterns, with high responses concentrated around the robot arms and manipulated objects while background regions remain stable. This suggests that Token-World captures semantically meaningful action-conditioned changes rather than accumulating global token drift.



(a) Simulator step time comparison for the simulation-interface evaluation. Token-World directly predicts VLA visual tokens, avoiding RGB generation and re-encoding.



(b) RMSE over rollout steps across four RoboTwin [10] tasks. Token-World is compared with WorldGym [38] and DINO-WM [56] under action-replay rollout.

Figure 4: **Efficiency and long-horizon rollout fidelity.** Left: Token-World achieves lower simulator step time than RGB-based world-model baselines. Right: Token-World maintains lower RMSE growth over long-horizon rollouts compared with WorldGym [38] and DINO-WM [56].

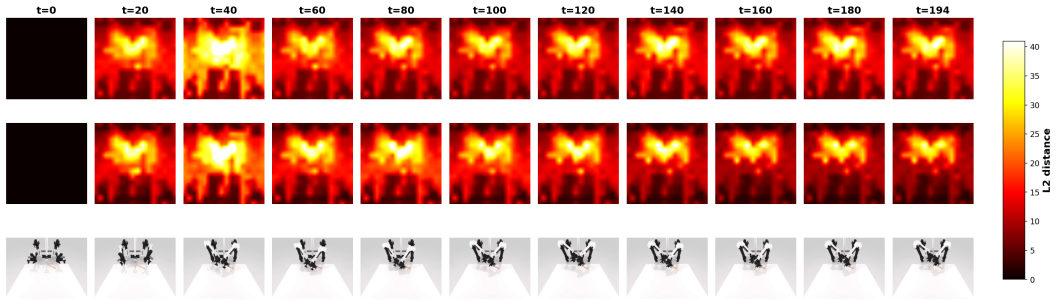


Figure 5: **Visualization of semantic token change.** We visualize per-patch semantic change from the initial frame, measured by the L2 distance in the Paligemma visual-embedding space and upsampled from a 16×16 patch grid. Top: predicted token rollout. Middle: ground-truth token trajectory. Bottom: corresponding RGB frames. Brighter regions indicate larger semantic deviation from the initial visual state.

6 Conclusion

We introduced Token-World, a VLA-oriented world-model simulator that rolls out future states directly in the policy-facing visual-token space. Unlike RGB-based simulators that reconstruct future images and then re-encode them into VLA inputs, Token-World avoids the decode–encode interface and directly models the representation consumed by downstream policies. To this end, Token-World combines a spatio-temporal multimodal Transformer, an x_0-parameterized flow-matching objective, and a recurrent state to capture high-dimensional token dynamics and preserve long-horizon interaction context.

Experiments on RoboTwin manipulation tasks validate Token-World along both dynamics-modeling and simulation-interface axes. Compared with deterministic token predictors and RGB-based world-model simulators, Token-World achieves better token prediction fidelity, lower simulator step time, and more reliable rollout-based success classification. These results suggest that aligning the simulation space with the downstream VLA policy is an important design principle for efficient and scalable world modeling. Future work will extend direct token-space simulation from action-replay evaluation to closed-loop policy evaluation, synthetic data generation, and policy learning for VLA agents.

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel De Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901, 2023.

[2] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, et al. V-jepa 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.

[3] Federico Baldassarre, Marc Szafraniec, Basile Terver, Vasil Khalidov, Francisco Massa, Yann LeCun, Patrick Labatut, Maximilian Seitzer, and Piotr Bojanowski. Back to the features: Dino as a foundation for video world models. *arXiv preprint arXiv:2507.19468*, 2025.

[4] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *arXiv preprint arXiv:2404.08471*, 2024.

[5] Irwan Bello, Hieu Pham, Quoc V Le, Mohammad Norouzi, and Samy Bengio. Neural combinatorial optimization with reinforcement learning. *arXiv preprint arXiv:1611.09940*, 2016.

[6] Lucas Beyer, Andreas Steiner, André Susano Pinto, Alexander Kolesnikov, Xiao Wang, Daniel Salz, Maxim Neumann, Ibrahim Alabdulmohsin, Michael Tschannen, Emanuele Bugliarello, et al. Paligemma: A versatile 3b vlm for transfer. *arXiv preprint arXiv:2407.07726*, 2024.

[7] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0: A vision-language-action flow model for general robot control, 2026. URL `https://arxiv.org/abs/2410.24164`.

[8] Akshay L Chandra, Iman Nematollahi, Chenguang Huang, Tim Welschehold, Wolfram Burgard, and Abhinav Valada. Diwa: Diffusion policy adaptation with world models. *arXiv preprint arXiv:2508.03645*, 2025.

[9] Boyuan Chen, Diego Martí Monsó, Yilun Du, Max Simchowitz, Russ Tedrake, and Vincent Sitzmann. Diffusion forcing: Next-token prediction meets full-sequence diffusion. *Advances in Neural Information Processing Systems*, 37:24081–24125, 2024.

[10] Tianxing Chen, Zanxin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, Qiwei Liang, Xianliang Lin, Yiheng Ge, Zhenyu Gu, et al. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025.

[11] Kyunghyun Cho, Bart Van Merriënboer, Çağlar Gulçehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder–decoder for statistical machine translation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1724–1734, 2014.

[12] Mostafa Dehghani, Josip Djolonga, Basil Mustafa, Piotr Padlewski, Jonathan Heek, Justin Gilmer, Andreas Peter Steiner, Mathilde Caron, Robert Geirhos, Ibrahim Alabdulmohsin, et al. Scaling vision transformers to 22 billion parameters. In *International conference on machine learning*, pages 7480–7512. PMLR, 2023.

[13] Marc Peter Deisenroth, Dieter Fox, and Carl Edward Rasmussen. Gaussian processes for data-efficient learning in robotics and control. *IEEE transactions on pattern analysis and machine intelligence*, 37(2):408–423, 2013.

[14] Chelsea Finn and Sergey Levine. Deep visual foresight for planning robot motion. In *2017 IEEE international conference on robotics and automation (ICRA)*, pages 2786–2793. IEEE, 2017.

[15] Kevin Frans, Danijar Hafner, Sergey Levine, and Pieter Abbeel. One step diffusion via shortcut models. *arXiv preprint arXiv:2410.12557*, 2024.

[16] Shenyuan Gao, William Liang, Kaiyuan Zheng, Ayaan Malik, Seonghyeon Ye, Sihyun Yu, Wei-Cheng Tseng, Yuzhu Dong, Kaichun Mo, Chen-Hsuan Lin, et al. Dreamdojo: A generalist robot world model from large-scale human videos. *arXiv preprint arXiv:2602.06949*, 2026.

[17] Yanjiang Guo, Lucy Xiaoyang Shi, Jianyu Chen, and Chelsea Finn. Ctrl-world: A controllable generative world model for robot manipulation. *arXiv preprint arXiv:2510.10125*, 2025.

[18] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2(3):440, 2018.

[19] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. *arXiv preprint arXiv:1912.01603*, 2019.

[20] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *International conference on machine learning*, pages 2555–2565. PMLR, 2019.

[21] Danijar Hafner, Wilson Yan, and Timothy Lillicrap. Training agents inside of scalable world models. *arXiv preprint arXiv:2509.24527*, 2025.

[22] Haoran He, Yang Zhang, Liang Lin, Zhongwen Xu, and Ling Pan. Pre-trained video generative models as world simulators. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 4645–4653, 2026.

[23] Jonathan Ho, Nal Kalchbrenner, Dirk Weissenborn, and Tim Salimans. Axial attention in multidimensional transformers. *arXiv preprint arXiv:1912.12180*, 2019.

[24] Siqiao Huang, Jialong Wu, Qixing Zhou, Shangchen Miao, and Mingsheng Long. Vid2world: Crafting video diffusion models to interactive world models. *arXiv preprint arXiv:2505.14357*, 2025.

[25] Xun Huang, Zhengqi Li, Guande He, Mingyuan Zhou, and Eli Shechtman. Self forcing: Bridging the train-test gap in autoregressive video diffusion. *arXiv preprint arXiv:2506.08009*, 2025.

[26] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization, 2025. URL `https://arxiv.org/abs/2504.16054`.

[27] Joel Jang, Seonghyeon Ye, Zongyu Lin, Jiannan Xiang, Johan Bjorck, Yu Fang, Fengyuan Hu, Spencer Huang, Kaushil Kundalia, Yen-Chen Lin, et al. Dreamgen: Unlocking generalization in robot learning through video world models. *arXiv preprint arXiv:2505.12705*, 2025.

[28] Zhennan Jiang, Kai Liu, Yuxin Qin, Shuai Tian, Yupeng Zheng, Mingcai Zhou, Chao Yu, Haoran Li, and Dongbin Zhao. World4rl: Diffusion world models for policy refinement with reinforcement learning for robotic manipulation. *arXiv preprint arXiv:2509.19080*, 2025.

[29] Zhennan Jiang, Shangqing Zhou, Yutong Jiang, Zefang Huang, Mingjie Wei, Yuhui Chen, Tianxing Zhou, Zhen Guo, Hao Lin, Quanlu Zhang, et al. Wovr: World models as reliable simulators for post-training vla policies with rl. *arXiv preprint arXiv:2602.13977*, 2026.

[30] Efstathios Karypidis, Ioannis Kakogeorgiou, Spyros Gidaris, and Nikos Komodakis. Dino-foresight: Looking into the future with dino. *arXiv preprint arXiv:2412.11673*, 2024.

[31] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[32] Tianhong Li and Kaiming He. Back to basics: Let denoising generative models denoise. *arXiv preprint arXiv:2511.13720*, 2025.

[33] Yaxuan Li, Yichen Zhu, Junjie Wen, Chaomin Shen, and Yi Xu. Worldeval: World model as real-world robot policies evaluator. *arXiv preprint arXiv:2505.19017*, 2025.

[34] Yue Liao, Pengfei Zhou, Siyuan Huang, Donglin Yang, Shengcong Chen, Yuxin Jiang, Yue Hu, Jingbin Cai, Si Liu, Jianlan Luo, et al. Genie envisioner: A unified world foundation platform for robotic manipulation. *arXiv preprint arXiv:2508.05635*, 2025.

[35] Zeyi Liu, Shuang Li, Eric Cousineau, Siyuan Feng, Benjamin Burchfiel, and Shuran Song. Geometry-aware 4d video generation for robot manipulation. *arXiv preprint arXiv:2507.01099*, 2025.

[36] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. Leworld-model: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.

[37] Lorenzo Mur-Labadia, Matthew Muckley, Amir Bar, Mido Assran, Koustuv Sinha, Mike Rabbat, Yann LeCun, Nicolas Ballas, and Adrien Bardes. V-jepa 2.1: Unlocking dense features in video self-supervised learning. *arXiv preprint arXiv:2603.14482*, 2026.

[38] Julian Quevedo, Ansh Kumar Sharma, Yixiang Sun, Varad Suryavanshi, Percy Liang, and Sherry Yang. Worldgym: World model as an environment for policy evaluation. *arXiv preprint arXiv:2506.00613*, 2025.

[39] Ansh Kumar Sharma, Yixiang Sun, Ninghao Lu, Yunzhe Zhang, Jiarao Liu, and Sherry Yang. World-gymnast: Training robots with reinforcement learning in a world model. *arXiv preprint arXiv:2602.02454*, 2026.

[40] Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

[41] Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim GJ Rudner, and Yann LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models. *arXiv preprint arXiv:2502.14819*, 2025.

[42] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[43] Jingwen Sun, Wenyao Zhang, Zekun Qi, Shaojie Ren, Zezhi Liu, Hanxin Zhu, Guangzhong Sun, Xin Jin, and Zhibo Chen. Vla-jepa: Enhancing vision-language-action model with latent world model. *arXiv preprint arXiv:2602.10098*, 2026.

[44] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.

[45] GigaWorld Team, Angen Ye, Boyuan Wang, Chaojun Ni, Guan Huang, Guosheng Zhao, Haoyun Li, Jiagang Zhu, Kerui Li, Mengyuan Xu, et al. Gigaworld-0: World models as data engine to empower embodied ai. *arXiv preprint arXiv:2511.19861*, 2025.

[46] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.

[47] Wei-Cheng Tseng, Jinwei Gu, Qinsheng Zhang, Hanzi Mao, Ming-Yu Liu, Florian Shkurti, and Lin Yen-Chen. Scalable policy evaluation with video world models. *arXiv preprint arXiv:2511.11520*, 2025.

[48] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[49] Yixuan Wang, Rhythm Syed, Fangyu Wu, Mengchao Zhang, Aykut Onol, Jose Barreiros, Hooshang Nayyeri, Tony Dear, Huan Zhang, and Yunzhu Li. Interactive world simulator for robot policy training and evaluation. *arXiv preprint arXiv:2603.08546*, 2026.

[50] John Won, Kyungmin Lee, Huiwon Jang, Dongyoung Kim, and Jinwoo Shin. Dual-stream diffusion for world-model augmented vision-language-action model. *arXiv preprint arXiv:2510.27607*, 2025.

[51] Philipp Wu, Alejandro Escontrela, Danijar Hafner, Pieter Abbeel, and Ken Goldberg. Daydreamer: World models for physical robot learning. In *Conference on robot learning*, pages 2226–2240. PMLR, 2023.

[52] Jiazhi Yang, Kunyang Lin, Jinwei Li, Wencong Zhang, Tianwei Lin, Longyan Wu, Zhizhong Su, Hao Zhao, Ya-Qin Zhang, Li Chen, et al. Rise: Self-improving robot policy with compositional world model. *arXiv preprint arXiv:2602.11075*, 2026.

[53] Sherry Yang, Yilun Du, Kamyar Ghasemipour, Jonathan Tompson, Leslie Kaelbling, Dale Schuurmans, and Pieter Abbeel. Learning interactive real-world simulators. *arXiv preprint arXiv:2310.06114*, 2023.

[54] Biao Zhang and Rico Sennrich. Root mean square layer normalization. *Advances in neural information processing systems*, 32, 2019.

[55] Haoyu Zhen, Qiao Sun, Hongxin Zhang, Junyan Li, Siyuan Zhou, Yilun Du, and Chuang Gan. Tesseract: learning 4d embodied world models. *arXiv preprint arXiv:2504.20995*, 2025.

[56] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. Dino-wm: World models on pre-trained visual features enable zero-shot planning. *arXiv preprint arXiv:2411.04983*, 2024.

[57] Fangqi Zhu, Zhengyang Yan, Zicong Hong, Quanxin Shou, Xiao Ma, and Song Guo. Wmpo: World model-based policy optimization for vision-language-action models. *arXiv preprint arXiv:2511.09515*, 2025.

A Task and Dataset Details

We conduct all experiments on four simulated manipulation tasks from the RoboTwin [10] benchmark, instantiated on the Aloha-AgileX platform, a 14-DoF dual-arm robot in simulation. These tasks cover diverse bimanual coordination patterns and contact dynamics. The official RoboTwin task descriptions are as follows:

- **Grab-Roller**: Use both arms to grab the roller on the table.
- **Place-Container-Plate**: Place the container onto the plate.
- **Place-Burger-Fries**: Use both arms to pick the hamburger and french fries and place them onto the tray.
- **Stack-Bowls-Two**: Stack the two bowls on top of each other.

At each time step, the control input is represented as a 14-dimensional action vector obtained by concatenating the commands of the two arms, including both end-effector motion and gripper control. The proprioceptive state is represented using absolute end-effector states under the same per-arm parameterization, also resulting in a 14-dimensional vector. This shared representation enables a consistent interface between action and proprioceptive inputs during model training and rollout.

Table 3 summarizes the number of successful and failed episodes and their average trajectory lengths. Moreover, the train/validation split for successful and failed trajectories is reported in Table 4. During preprocessing, we compute the mean and standard deviation of the action and proprioceptive channels on the training split only, and use these statistics to normalize both training and validation data.

B Compute Resources, Hyperparameters and Implementation Details

In this section, we summarize the compute resources, hyperparameters and implementation details used in our experiments. Unless otherwise specified, all tasks share the same model architecture and optimization setup.

Table 3: Per-task dataset statistics.

Task	# Succ.	Succ. Len.	# Fail	Fail Len.
Grab-Roller	38	93.95	50	399.00
Place-Container-Plate	50	241.86	50	499.00
Place-Burger-Fries	50	158.54	50	399.00
Stack-Bowls-Two	50	313.54	50	899.00

Table 4: Train/validation split for successful and failed episodes.

Task	Train		Validation	
	Succ.	Fail	Succ.	Fail
Grab-Roller	30	40	8	10
Place-Container-Plate	40	40	10	10
Place-Burger-Fries	40	40	10	10
Stack-Bowls-Two	40	40	10	10

B.1 Compute Resources and Hyperparameters

All Token-World models are trained separately for each RoboTwin task. Unless otherwise specified, the model architecture uses hidden dimension 2048, 8 Transformer layers, 16 attention heads, head dimension 256, and 64 maximum denoising steps. Training uses batch size 8, Adam optimizer, learning rate 3.0×10^{-5}, and 100 epochs.

For each task, Token-World is trained with bfloat16 mixed precision for approximately 12 wall-clock hours on 8 NVIDIA A100 GPUs, corresponding to roughly 96 GPU-hours. Baseline models are trained under matched data and training-budget settings whenever applicable. Inference step time is measured on the same hardware for all methods. For RGB-based baselines, the reported step time includes both RGB generation and re-encoding with the frozen VLA visual tokenizer; for Token-World, it measures direct VLA visual-token generation. All timing results are averaged over validation rollouts. The architecture and training hyperparameters are summarized in Table 5.

B.2 Implementation Details

Since adjacent frames often differ only slightly, directly modeling every frame transition introduces substantial temporal redundancy. Following DINO-WM [56], we therefore introduce a frameskip parameter k. Concretely, given the observation at time step t, the model conditions on the sequence of k intermediate actions and directly predicts the observation at time step $t + k$. We use the *same* frameskip setting for DINO-WM and our method on each task for a fair comparison. We use a frameskip of 2 for the two shorter-horizon tasks, *Grab-Roller* and *Place-Container-Plate*, and a frameskip of 3 for the longer-horizon tasks, *Place-Burger-Fries* and *Stack-Bowls-Two*.

For action tokenization, we further split the 14-dimensional control vector by actuator type. Dimensions corresponding to delta end-effector motion are treated as continuous and encoded as a weighted sum of type-specific embeddings, i.e., $\sum_i \text{embed}_i \cdot \text{action}_i$, while the gripper dimensions are treated as discrete and encoded with learned embedding lookups. Thus, the action space remains a unified 14-dimensional control vector, but different subcomponents are tokenized differently according to their semantics. Proprioception is represented separately as a proprioceptive token. The predictor is implemented as an axial time-space Transformer.

C Decoder Architecture and Visual Metrics

C.1 Decoder Architecture and Training Details

To reconstruct pixel-space observations from visual tokens, we employ a **convolutional decoder with residual blocks and bottleneck self-attention**. Starting from low-resolution token features, the decoder progressively recovers spatial structure and appearance through residual convolution and hierarchical upsampling.

Table 5: Architecture and training hyperparameters. All tasks share the same setup except for task-dependent frameskip.

Model architecture

Parameter	Value
Hidden dimension	2048
Depth	8
Attention heads	16
Head dimension	256
Max denoise steps	64

Training hyperparameters

Parameter	Value
Batch size	8
Epochs	100
Learning rate	3.0×10^{-5}
Optimizer	Adam

Table 6: Decoder architecture and training hyperparameters.

Architecture hyperparameters

Parameter	Value
Input embedding dimension	2048
Base channel width	512
Channel multipliers	[1, 1, 2, 2, 4]
ResNet blocks per level	2
Output resolution	224×224
GroupNorm groups	32
Middle-block attention	True
Attention resolution	16
Output channels	3

Training hyperparameters

Parameter	Value
Batch size	8
Learning rate	3×10^{-4}
Learning rate schedule	Cosine annealing
Minimum learning rate	1×10^{-6}
Number of epochs	25
Latent loss weight	0.25

At the bottleneck resolution, the decoder applies an input convolution followed by two residual convolutional blocks with a self-attention block inserted between them. Each residual block contains two *GroupNorm* $\rightarrow$ *SiLU* $\rightarrow 3 \times 3$ *convolution* layers together with a skip connection; when the input and output channel dimensions differ, the skip path uses a 1×1 convolutional projection for dimension matching. The self-attention block is applied only at the bottleneck resolution to model global spatial dependencies before decoding to higher resolutions.

The bottleneck features are then passed through four decoding stages. Each stage contains two residual blocks followed by a $2\times$ spatial upsampling operation implemented by nearest-neighbor interpolation and a 3×3 convolution. Across these stages, the spatial resolution is progressively increased, while the channel dimension is reduced from 2048 to 512.

The output head uses GroupNorm, SiLU, and a final 3×3 convolution to produce a 3-channel RGB image, followed by a `tanh` activation. The architecture hyperparameters and training hyperparameters are summarized in Table 6.

C.2 Qualitative Results and Visual Metrics

We further evaluate the decoder both qualitatively and quantitatively. Figure 6 shows representative reconstruction results on validation trajectories. For each example, the first row presents the ground-truth frames, and the second row shows the images reconstructed from latent visual tokens.

For quantitative evaluation, we report standard visual reconstruction metrics in Table 7. In addition to distortion-based metrics such as PSNR and SSIM, we also report the perceptual metric LPIPS to better reflect visual fidelity.

D Latent Success Classifier Details

In this section, we provide the implementation and training details for the latent success classifier. Crucially, aligning with the core motivation of Token-World, our success classifier operates *directly* on the VLA visual token representations rather than raw RGB pixels. This design avoids the computational overhead of decoding imagined latents back to images and yields a highly efficient evaluator.

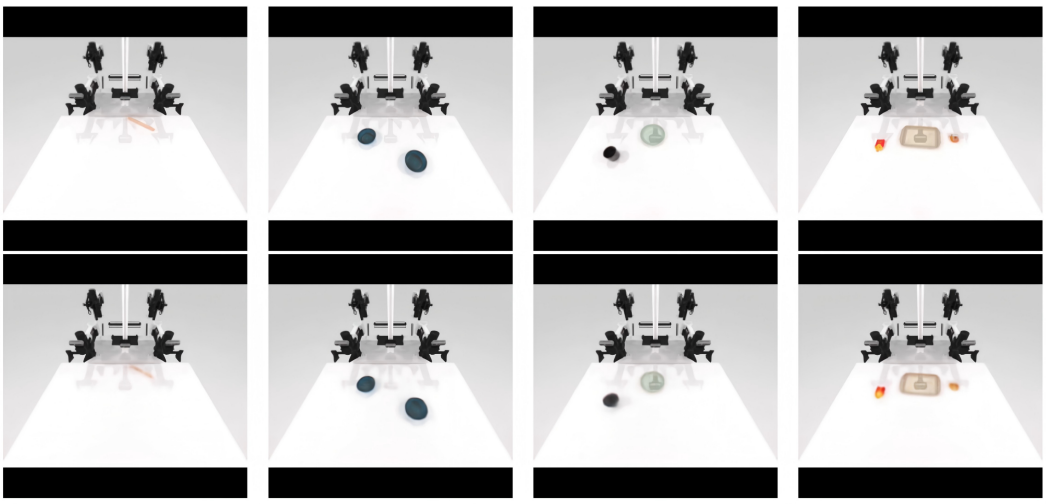


Figure 6: **Qualitative reconstruction results of the decoder.** Representative examples from the four RoboTwin tasks. The first row shows the ground-truth frames, and the second row shows the corresponding images reconstructed from latent visual tokens.

Table 7: **Visual reconstruction metrics of the decoder on the validation set.** All metrics are computed over the full validation set, containing 29,850 frames in total. Higher PSNR/SSIM and lower LPIPS are better.

Task	PSNR↑	SSIM↑	LPIPS↓
Grab-Roller	35.13	0.9874	0.0121
Place-Container-Plate	36.02	0.9880	0.0115
Place-Burger-Fries	36.39	0.9893	0.0125
Stack-Bowls-Two	36.10	0.9878	0.0102
Average	35.91	0.9881	0.0116

D.1 Architecture and Data Processing

The latent success classifier is a lightweight multi-layer perceptron (MLP) with approximately 1.1M parameters. It takes the pre-extracted VLA token latents of shape 256×2048 (number of patches $\times$ embedding dimension) as input. The network first applies spatial mean pooling across the patch dimension to aggregate the features into a single 2048-dimensional global representation. This is followed by a 3-layer MLP ($2048 \to 512 \to 128 \to 1$) with Layer Normalization and ReLU activations. A final Sigmoid activation outputs the success probability $p \in [0, 1]$. The entire model contains only $\sim$1.1M trainable parameters, making it extremely fast for both training and inference.

To train the classifier, we utilize the same four manipulation tasks from the RoboTwin benchmark. We split the episodes into 85% for training and 15% for validation at the trajectory level to prevent data leakage. For successful trajectories, we label the last 20% of frames as positive samples (success, $y = 1$) and the first 80% as negative samples (in-progress, $y = 0$). For failure trajectories, all frames are labeled as negative. To address class imbalance, we dynamically sample frames during training and apply a $3.0\times$ positive sample weight in the loss function.

D.2 Training Details

The model is trained using Binary Cross-Entropy (BCE) loss. We use the AdamW optimizer with a cosine annealing learning rate schedule following a brief linear warmup. Detailed hyperparameters are summarized in Table 8.

Table 8: **Latent success classifier training hyperparameters.** Configuration for training the success classifier directly on VLA visual tokens.

Hyperparameter	Value
Input shape	256×2048 VLA token latents
Optimizer	AdamW
Learning rate	1×10^{-4}
Weight decay	1×10^{-4}
Batch size	64
Training epochs	30
Learning rate schedule	Cosine annealing with 2-epoch linear warmup
Positive sample weight	3.0

D.3 Validation Performance

Despite having approximately $10\times$ fewer parameters than a standard ResNet-18 pixel-based classifier, the latent success classifier demonstrates strong discriminative capability. It achieves its best validation performance at epoch 28. As shown in Table 9, the model balances precision and recall effectively, yielding an F1 score of 90.67% and a high recall of 93.38%, ensuring reliable and fast success-detection signals during Token-World policy evaluation.

Table 9: **Latent success classifier validation results.** Binary classification metrics evaluated on the held-out validation set with 5,194 latent samples.

Accuracy	Precision	Recall	F1 Score
95.48%	88.12%	93.38%	90.67%

E Assets and Licenses

We use existing research assets only for simulation, tokenization, and baseline comparison. RoboTwin is used as the simulated manipulation benchmark and is released under the MIT license. The VLA visual tokenizer is based on PaliGemma/SigLIP components: PaliGemma access requires agreeing to Google's usage license, and the public SigLIP model card lists an Apache-2.0 license. We also use or compare against existing research codebases and models, including π_0/OpenPI, DINO-WM, WorldGym, and Ctrl-World, following their intended research use. We cite all external assets in the main paper and do not redistribute their datasets, model weights, or simulator assets as part of this submission.

F Broader Impacts and Limitations

Token-World studies how learned world models can simulate future robot states more efficiently in the representation space consumed by downstream VLA policies. A potential positive impact is to reduce the cost of embodied learning: if imagined rollouts can be generated directly in a policy-facing representation space, they may support faster policy evaluation, data filtering, and future policy improvement with fewer physical robot trials. This can be especially useful in robotics, where real-world data collection is expensive, slow, and may cause hardware wear.

At the same time, learned simulators can introduce risks when their predictions are treated as reliable substitutes for real interaction. A world model may produce plausible but incorrect rollouts, and such errors may bias downstream policy evaluation or data generation. This risk is particularly important for long-horizon manipulation, where small prediction errors can compound over time. In this work, we therefore evaluate Token-World under action-replay rollouts and rollout-based success prediction, rather than claiming full closed-loop VLA policy improvement.

Token-World is currently evaluated in simulation on RoboTwin manipulation tasks. The conclusions should not be interpreted as evidence that direct token-space simulation is sufficient for safe real-world

robot deployment. Before being used for policy optimization or real-robot decision making, such simulators should be validated under closed-loop settings, tested across broader task distributions, and combined with uncertainty estimation or real-world verification. We view this work as a step toward more efficient VLA-oriented world modeling, rather than a complete replacement for physical evaluation.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive" or "we were unable to find the license for the dataset we used"). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading "NeurIPS Paper Checklist"**,
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers**.

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction state the main scope of the paper: direct VLA visual-token simulation, dynamics modeling, and action-replay evaluation. The claims are supported by the experiments in Sec. 5.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

Justification: We discuss the main limitation that our evaluation focuses on action-replay rollouts rather than full closed-loop VLA policy improvement, and identify closed-loop evaluation and policy learning as future work in Sec. 6.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper does not present new theoretical theorems or formal proofs. The information-theoretic discussion is used to motivate the decode–encode bottleneck rather than as a standalone theoretical result.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

Justification: The paper describes the model architecture, training objective, rollout protocol, datasets, baselines, metrics, and implementation details needed to reproduce the main experimental results; additional hyperparameters are provided in the Appendix B.2.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: Code and processed data are not released at submission time due to the anonymous review process and dependency on simulator assets. We provide detailed implementation, data split, and hyperparameter descriptions to support reproducibility.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.

- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: We specify the task setup, train/validation splits, baselines, rollout protocol, metrics, model hyperparameters, optimizer settings, and training details in Sec. 5 and the Appendix A and B.2.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [No]

   Justification: We do not report error bars because training all world-model baselines across multiple seeds is computationally expensive. Instead, results are averaged over held-out validation trajectories and reported consistently across tasks.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
   - If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: We report model architecture, training hyperparameters, inference step time, hardware setup, and approximate training cost in Appendix B.1 and Sec. 5.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: [Yes]

Justification: The research uses simulated robot manipulation data and existing public models/datasets, and does not involve human subjects or sensitive personal data. We have reviewed the NeurIPS Code of Ethics and do not identify violations.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: We discuss potential positive impacts of more efficient learned simulators for robot learning, as well as limitations and risks related to using imperfect imagined rollouts for downstream policy evaluation or training in Appendix F.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.

- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The paper does not release high-risk pretrained language models, image generators, scraped datasets, or models intended for public deployment.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: We cite the datasets, models, and codebases used in the paper, including RoboTwin, π_0/OpenPI, PaliGemma/SigLIP, DINO-WM, WorldGym, and Ctrl-World. Asset usage and license information are summarized in Appendix E.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: The paper does not introduce or release a new dataset or benchmark. If code or model checkpoints are released, they will be documented separately with usage instructions.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing experiments or research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper does not involve human subjects, so IRB approval or equivalent review is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: LLMs are not used as an original or non-standard component of the core method. Existing VLA or VLM used for tokenization are cited as existing model assets.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.